load_balancer_stepwise_plan

# Load Balancer — Incremental Build Plan (first-timer pace)

**Approach: incremental, phase-gated.** Each step below must produce something you can *run and demo* before you move to the next. Never start step N+1 with step N half-working — that's how bugs from two different layers get confused with each other.

---

## Step 0 — Environment + "hello socket" (budget: 1–2 sessions)

**Do:**

1. Confirm a Linux ekvironment (native or WSL2) with g++/clang and `make` or `cmake` working.
2. Read Beej's Guide to Network Programming — just the sections on TCP client and TCP server (skip UDP, skip advanced sections for now).
3. Write two tiny programs by hand, not copy-pasted: a server that `socket()` → `bind()` → `listen()` → `accept()` s one connection and prints what it receives, and a client that `socket()` → `connect()` s and sends one string.

**Done when:** your client and server exchange one message on localhost and you can explain, out loud, what each syscall does — not just that the code works.

**Why this matters:** every bug you'll hit later (connection resets, half-closed sockets, EAGAIN) traces back to not having a solid mental model of what these four syscalls actually do at the kernel level. Rushing this step is the #1 cause of confused debugging in Phase 1–2.

---

## Step 1 — Dumb single-backend proxy (budget: 2–3 sessions)

**Do:**

1. Write a “backend”: the simplest possible TCP echo server, or just run `nc -l 9000` in another terminal as a fake backend.
2. Write your proxy: `accept()` a client, `connect()` to the one hardcoded backend, then relay bytes in both directions until either side closes.
3. Simplest relay implementation: two threads per connection (one thread reads client→backend, the other reads backend→client), or a single thread doing blocking reads/writes in a loop if you want to start even simpler before adding threads.

**Key decision to make explicitly (don’t skip):** how do you detect “connection closed” (recv returns 0) vs “connection error” (recv returns -1, check errno)? Write this down as a comment in your code — you’ll need this exact logic in every later phase.

**Done when:** `curl` or `nc` through your proxy to your one backend gets a correct response, and you can kill the backend mid-request and watch your proxy *not crash*.

---

# Step 2 — Multi-backend + round robin + thread-per-connection (budget: 2 sessions)

**Do:**

1. Replace the hardcoded backend with a list (even a hardcoded `std::vector` is fine for now — config file comes later).
2. Add a round-robin counter: `next = counter++ % backends.size()`.
3. Move connection handling into `std::thread` per accepted client (thread-per-connection model).

**Done when:** repeated requests visibly rotate across your backends (print which backend each connection went to).

**Why round robin first, not least-connections:** round robin needs no shared mutable state between threads. Least-connections does (a shared counter). Get the threading model *working and stable* on the simple algorithm before you introduce your first real concurrency bug surface.

---

# Step 3 — Decision point: config + logging (budget: 1 session)

**Do:**

1. Add a minimal config (command-line flags are enough — backend list, algorithm choice).
2. Add basic stats: requests routed per backend, printed on exit or on a signal.

**Done when:** you can change the backend list or algorithm without recompiling.

**Why now, not later:** you're about to add 2 more algorithms and health checks. Without switchable config, you'll be hand-editing code for every test, which multiplies debugging time.

---

# Step 4 — Least connections (budget: 1–2 sessions)

**Do:**

1. Add a per-backend active-connection counter: increment on connect, decrement on disconnect.
2. Protect it with a `std::mutex` (don't reach for atomics or lock-free structures yet — mutex is correct and sufficient at this scale).
3. Route to whichever backend has the lowest count at the moment of routing.

**Done when:** under concurrent load (open several connections at once), connection counts across backends stay roughly even.

**This is your first real concurrency bug encounter.** Expect to hit a race condition here even if your logic looks right on paper — that's normal and is the actual learning objective of this step, not a sign you did something wrong.

---

# Step 5 — Consistent hashing (budget: 2 sessions, include reading time)

**Do:**

1. Before writing code: find a visual “consistent hashing explained” blog post (not the original paper yet) and actually work through the ring diagram on paper.
2. Implement: hash backend IDs onto a numeric ring, hash the incoming client IP the same way, route to the next backend clockwise on the ring.

**Done when:** the same client IP consistently lands on the same backend across multiple connections, and adding/removing a backend only reshuffles a small fraction of existing mappings (verify this by printing before/after routing tables).

---

## Step 6 — Health checks + failover (budget: 2 sessions)

**Do:**

1. Background thread that pings each backend every N seconds (a lightweight TCP connect attempt is enough).
2. Shared “backend status” table (up/down), guarded by the same mutex pattern from Step 4.
3. Algorithm module skips “down” backends when routing.

**Done when:** you can kill a backend process mid-test, watch traffic stop routing to it within ~N seconds, then restart it and watch traffic resume.

**This is your demo moment** — the proposal’s “deliverable” list explicitly calls out a live deliberate-failure demo. Make sure this step is rock solid, not just “works once.”

---

## Step 7 — epoll rewrite (budget: 3–4 sessions — the hardest phase, budget generously)

**Honest flag:** this is the phase most likely to blow your time budget. Read `man 2 epoll` (or the Kerrisk chapter) *right before* starting this, not earlier — it won’t make sense in the abstract without a working blocking version to compare it against, which you now have.

**Do:**

1. Rewrite the connection handler as a single-threaded event loop: register client and backend sockets with `epoll`, loop on `epoll_wait`, only read/write sockets that are actually ready.
2. Keep your thread-per-connection version in a separate branch — don't delete it. If epoll runs out of time, you demo the working version instead of a broken one.

**Done when:** epoll version passes the same manual tests as Step 1–2, and you can show it handling more concurrent connections without spawning more threads (a simple `wrk -c 500` vs `wrk -c 5000` comparison against thread count is a good demo).

**If you're tight on time:** this is the step to cut or timebox hardest. A correct, well-tested thread-per-connection LB with all 3 algorithms and health checks is a complete, gradeable project on its own. An unfinished, buggy epoll rewrite is worse than not attempting it.

---

## Step 8 — Benchmark harness + nginx comparison (budget: 2–3 sessions, start in parallel with Step 6 if possible)

**Do:**

1. Configure nginx's `upstream` module with the *identical* backend pool, same algorithm (round robin, since both support it natively) — freeze this config before testing either system.
2. Write `wrk`-driving scripts that hit both, discarding a warm-up period, and record p50/p95/p99 latency plus throughput.
3. Raise `ulimit -n` before any load test, or your concurrency numbers will be artificially capped.
4. Deliberately kill a backend mid-benchmark run for both systems and measure failover recovery time.

**Done when:** you have real numbers/graphs for both systems under identical conditions — this is your project's actual empirical contribution, per your reframing with the professor.

---

## Step 9 — Report + polish (budget: 1–2 sessions)

**Do:** Write up methodology, raw data, and an honest discussion of *why* your numbers differ from nginx's — not just that they differ. This "why" is the difference between a demo and a result.

---

## If you run short on total time — cut in this order

1. Consistent hashing stretch behavior (keep the algorithm, drop deep churn analysis)
2. epoll rewrite (keep thread-per-connection as your submitted version)
3. Full benchmark sweep (keep one clean round-robin comparison over a full sweep of parameters)

Never cut: health checks + failover demo, and honest methodology in the report — these are what separates this from "just another sem project."